Spring MVC framework and Injection attack

Lecture notes

Goal of the chapter

In this chapter you will learn how to develop Java web applications by adhering to standard software engineering practices. The goal is for you to start writing robust application, which is a necessary step towards secure and reliable software.

Acknowledgement and further reading

- References:
- `https://docs.spring.io/spring-framework/`
- Book: Spring MVC Beginner's Guide, author: Amuthan Ganeshan

1 Spring MVC

Spring is now a long-time de-facto standard for Java enterprise software development. The framework was designed with developer productivity in mind, and it makes it easy to work with the existing Java and Java EE APIs. Using Spring, we can develop standalone applications, desktop applications, two-tier applications, web applications, distributed applications, enterprise applications, and so on.

Java, Spring and Hibernate form a popular combination to build client-server applications that are extensible, reliable and robust. Java is a general purpose programming languages; one of the most popular in the market when it comes to build large applications. Spring is a framework that helps developers to adhere to standard software engineering practices and streamline the development for applications which run on servers, for example web sites. Another framework with a similar goal is Jakarta EE (J2EE), but we are not going to focus on it in this introductory course.

Spring supports a Model-View-Controller (MVC) software engineer design pattern. This is the basic structure which most web applications are built on. The same is also true for mobile apps and desktop programs. Models, Views and

Controllers are distinctly different pieces of code that helps to provide different functions to your overall project. Because of that, they are kept separate and organized.

Some of the advantages of Spring MVC Framework are:

- **Separate roles** - The Spring MVC separates each role, where the model object, controller, command object, view resolver, DispatcherServlet, validator, etc. can be fulfilled by a specialized object.
- **Light-weight** - It uses light-weight servlet container to develop and deploy your application.
- **Powerful Configuration** - It provides a robust configuration for both framework and application classes that includes easy referencing across contexts, such as from web controllers to business objects and validators.
- **Rapid development** - The Spring MVC facilitates fast and parallel development.
- **Reusable business code** - Instead of creating new objects, it allows us to use the existing business objects.

Spring's web MVC framework is, like many other web MVC frameworks, request-driven, designed around a central Servlet that dispatches requests to controllers and offers other functionality that facilitates the development of web applications. Spring's `DispatcherServlet` however, does more than just that. It is completely integrated with the Spring IoC container and as such allows you to use every other feature that Spring has.

The request processing workflow of the Spring Web MVC `DispatcherServlet` is illustrated in the following diagram[1]. The pattern-savvy reader will recognize that the `DispatcherServlet` is an expression of the "Front Controller" design pattern (this is a pattern that Spring Web MVC shares with many other leading web frameworks).

The `DispatcherServlet` is an actual Servlet (it inherits from the `HttpServlet` base class), and as such is declared in the web.xml of your web application. You need to map requests that you want the DispatcherServlet to handle, by using a URL mapping in the same web.xml file. This is standard Java EE Servlet configuration; the following example shows such a `DispatcherServlet` declaration and mapping:

[1] Taken from `https://docs.spring.io/spring-framework/`

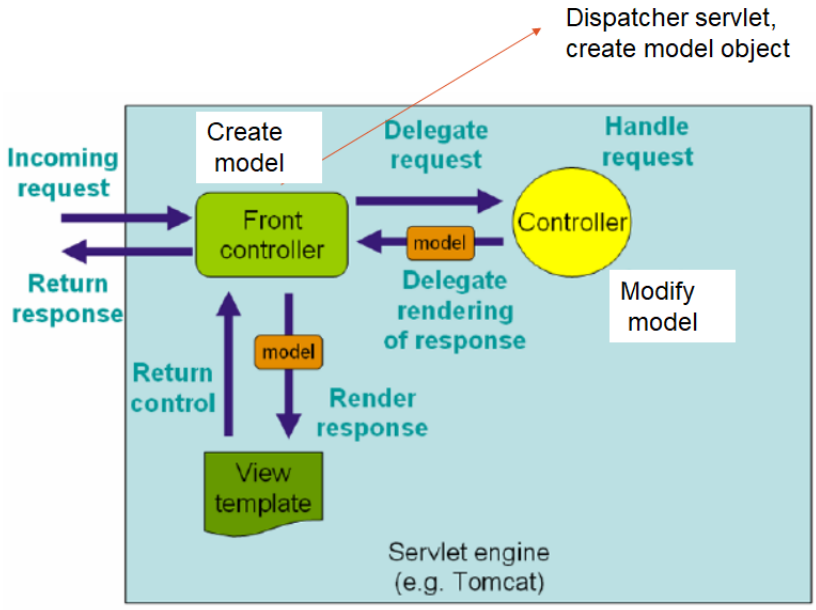

Figure 1: Spring Web Model-View-Controller. Source: `https://docs.spring.io/`

```
1
2  <web-app>
3
4      <servlet>
5          <servlet-name>example</servlet-name>
6          <servlet-class>org.springframework.web.servlet.
       DispatcherServlet</servlet-class>
7          <load-on-startup>1</load-on-startup>
8      </servlet>
9
10     <servlet-mapping>
11         <servlet-name>example</servlet-name>
12         <url-pattern>/example/*</url-pattern>
13     </servlet-mapping>
14
15 </web-app>
```

In the preceding example, all requests starting with /example will be handled by the DispatcherServlet instance named example. In a Servlet 3.0+ environment, you also have the option of configuring the Servlet container programmatically. Note that, the `DispatcherServlet` class has been already developed by the Spring team. Hence, as a developer, your task is to implement:

- **The models** - A model contains the data of the application, such as a database, web service, etc.

- **The controllers** - A controller contains the business logic of an application. This business logic is usually implemented on the server-side.
- **The views** - A view represents the provided information in a particular format, such as JSP, Thymelead, Groovy, Velocity, etc.

Hint

If you need to refresh the content on Servlets please go back to Chapter **??**.

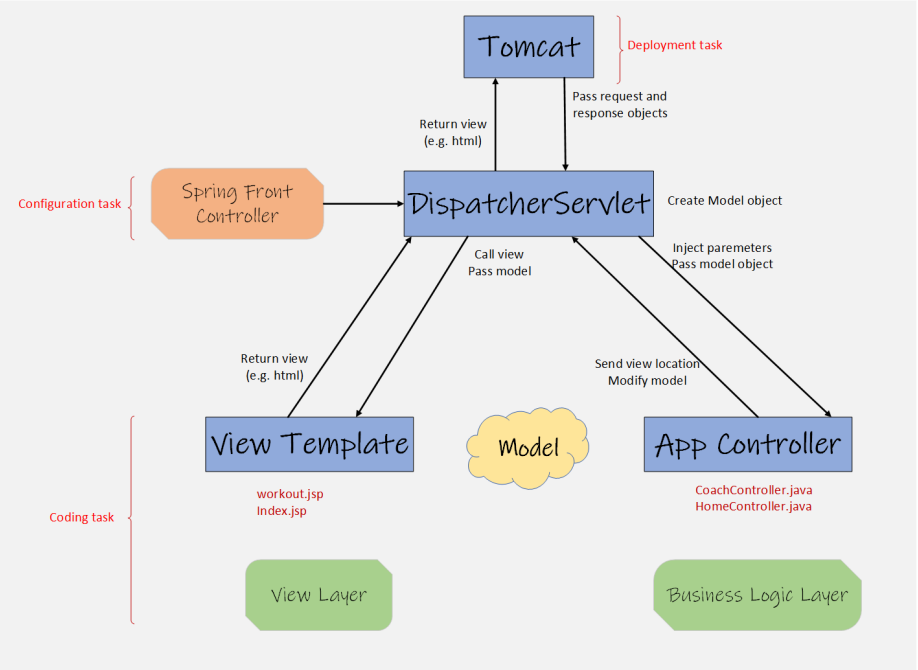

Figure 2: Architecture of our Spring-based web app.

2 Converting our coachweb application into a Spring project

As exciting as it was coding the coachweb app, we didn't follow best software engineering practices. Like most skills, internalising these practices takes time. Hence it is best to start using them with small projects so that they come natural once we work on large enterprise projects. Let us recall the Coach Servlet we implemented in our original project.

```
PrintWriter out = response.getWriter();
Random r = new Random();
if (r.nextBoolean())
 out.println("<HTML> <h1> No training today "
   + "</h1> </HTML>");
else
 out.println("<HTML> <h1> Spend 30 min running "
   + "</h1> </HTML>");
```

The above code is not a good practice of creating a web application, because we are mixing the business logic, i.e. tossing a coin etc., with the way we present the results, i.e. the use of `h1` font, etc. How can we split logic from presentation? Well, that's why we have are learning Spring. Based on the discussion we had earlier, these are the steps we need:

1. Configure the Spring MVC Dispatcher Servlet and set up a URL mapping to the Spring MVC Dispatcher Servlet. We will do that in the configuration file: WEB-INF/web.xml.
2. Create a *Controller* class the will implement our business logic. Remember that our web app is fairly simple. Our coach just toss a coin to tell us to either chill out or run for 30 min non-stop.
3. Create a JSP page to display the coach's message.
4. Tell Spring where to find our Controller class and JSP page.
5. Create a home page, exactly as the one we used in our original coachweb project. This will be the entry point to our app.
6. Deploy and execute!

2.1 Creating a new project and adding the Spring framework

The Spring MVC version of the application has been provided in your supplied VM as mentioned in Figure 3.

1. Let's create a new coachweb app. As we did before, we will create a new dynamic web project. This time we will call it `02-vul-coachwebapp`. Tell Eclipse to generate a content descriptor.
2. Download Spring Jar Files from `https://repo.spring.io/ui/`
   - Choose artifacts from the repo.
   - Find the jar files at libs-release/org/springframework/spring/5.3.9/spring-5.3.9-dist.zip
   - Unzip the file
   - Inside the folder you will find a folder titled 'libs', containing all jar files. Copy all those jar files to your lib folder located at src/main/webapp/WEB-INF/lib.

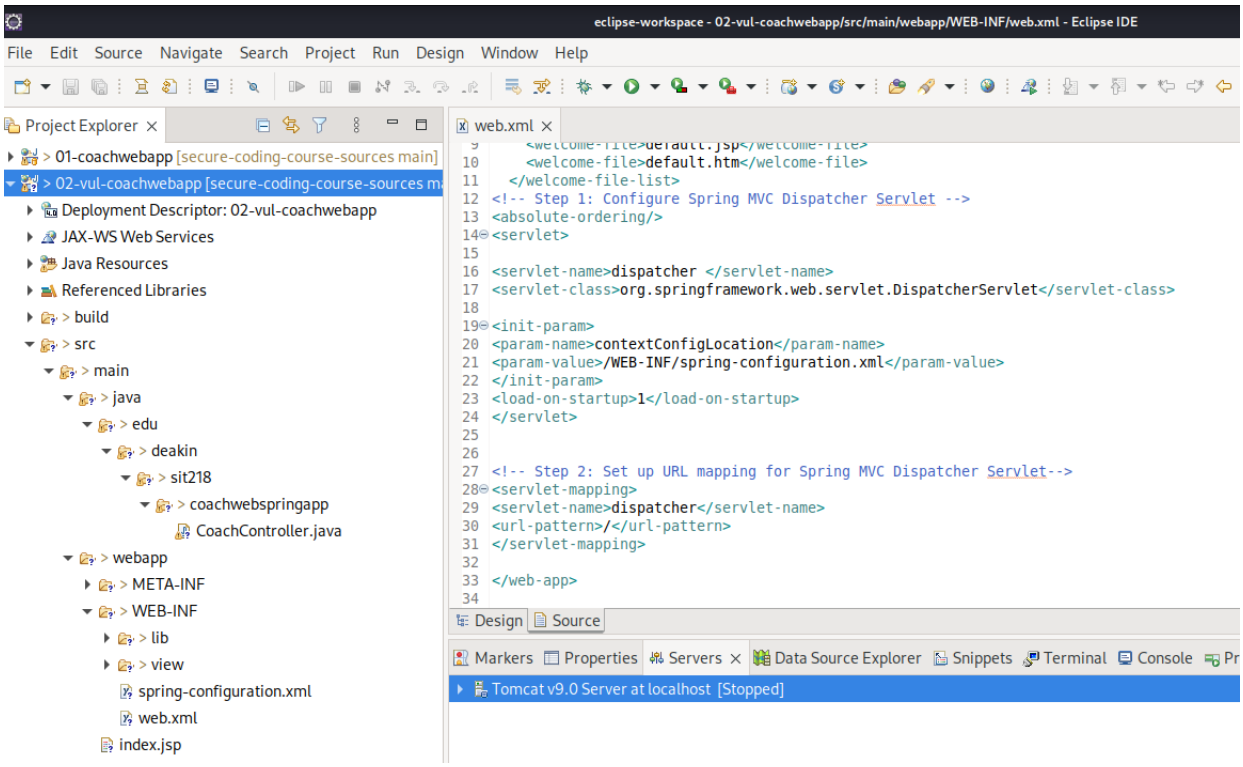

Figure 3: Spring MVC version of application

- Now go back to Eclipse and refresh the lib folder. All the jar files should now be identified by eclipse.

3. We also add the jar files necessary to build JSP pages. Those can be either found at `https://mvnrepository.com/artifact/javax.servlet.jsp.jstl/javax.servlet.jsp.jstl-api/1.2.1` and `https://mvnrepository.com/artifact/org.glassfish.web/javax.servlet.jsp.jstl/1.2.1` They are already have been supplied in the project of your VM

2.2 Configure the Spring MVC Dispatcher Servlet

Configuring the Spring MVC Dispatcher Servlet and setting up a URL mapping to the Spring MVC Dispatcher Servlet: To tell our web server that we will be using the Spring MVC Dispatcher Servlet as our Front Controller, we go the web.xml file and add the following.

```
<!-- Step 1: Configure Spring MVC Dispatcher Servlet -->
<absolute-ordering />

<servlet>
 <servlet-name>dispatcher</servlet-name>
 <servlet-class>org.springframework.web.servlet.DispatcherServlet</
    servlet-class>
 <init-param>
  <param-name>contextConfigLocation</param-name>
  <param-value>/WEB-INF/spring-configuration.xml</param-value>
 </init-param>
 <load-on-startup>1</load-on-startup>
</servlet>

<!-- Step 2: Set up URL mapping for Spring MVC Dispatcher Servlet -->
<servlet-mapping>
 <servlet-name>dispatcher</servlet-name>
 <url-pattern>/</url-pattern>
</servlet-mapping>
```

Two parts are worth paying attention in this file. First, the Spring dispatcher servlet will take control when the request has the pattern `/`. That essentially covers all requests. This has a consequence which we will address later. Second, we are telling the dispatcher servlet to use the context configuration file `/WEB-INF/spring-configuration.xml`. We haven't created that file yet. We will do so soon. This context configuration file will tell Spring where to find the controller, i.e. those implementing our business logic, and the views, i.e. those used to present the results to users.

2.3 Implementing our business logic be means of Controller classes

The second step is to create a *Controller* class the will implement our business logic. In our original coachweb application, this business logic was implemented a Servlet (`edu.deakin.sit218.coachwebapp.CoachServlet`). In Spring we don't longer use Servlets to implement business logic, but controllers. We create a controller by using the Java Annotation `@Controller`. Let's do this now.

The steps that we have used as below:

- Right click on the project and create a package `edu.deakin.sit218.coachwebspringapp`
- Right click on the package and create a class called `CoachController`
- Add the `@Controller` annotation. Your class should look as follows.

```java
package edu.deakin.sit218.coachwebspringapp;

import org.springframework.stereotype.Controller;

@Controller
public class CoachController {

}
```

- The next step is to give an URL pattern to this controller. Recall that such an URL pattern was "/workout" in our original app. Well, let's keep the same here. With a Spring Java Annotation we can do this very easily by using `@RequestMapping("/workout")`, which will place on top of the class name.

```java
package edu.deakin.sit218.coachwebspringapp;

import org.springframework.stereotype.Controller;
import org.springframework.web.bind.annotation.RequestMapping;

@Controller
@RequestMapping("/workout")
public class CoachController {

}
```

- The last step is to implement our business logic within this controller. We do so by adding the following method.

```java
@GetMapping
 public String workout(@RequestParam("studentName") String name,
      Model model) {


  model.addAttribute("name", name);

    Random r = new Random();
  if (r.nextBoolean())
   model.addAttribute("message", "No training today");
  else
   model.addAttribute("message", "Spend 30 min running");

return "workout";
}
```

Let's explain this method in detail.

  - The `@GetMapping` expresses that this method should be called once the URL pattern of the controller is found.
  - The argument `ModelMap model` will be injected by Spring. This object allows us to map attributes with values, which will later be used

by our JSP page to display the coach's message. Remember that the notion of *model* plays an important role in Spring MVC, as it allows controllers to add data to the model, which will later be accessed by our view templates. Simply put, the model can supply attributes used for rendering views.

- The method returns a string, which correspond to the name of the JSP page. That is, at some point we will need to create a `workout.jsp` page.

2.4 Create a JSP page to display the coach's message

- Right click on the WEB-INF folder and create a folder called *view*
- Inside the view folder create a JSP page called `workout.jsp`
- Add the following content:

```
<%@ page language="java" contentType="text/html; charset=ISO
    -8859-1"
    pageEncoding="ISO-8859-1"%>
<!DOCTYPE html>
<html>
<head>
<meta charset="ISO-8859-1">
<title>First spring webcoach-app</title>
</head>
<body>
<h1>Coach's Message</h1>
<p>Hi ${name}!</p>
<p>${message}</p>

</body>
</html>
```

- The preamble of this page indicates that this is a JSP page.
- The syntax `${message}` is used by the JSP language to access attribute values. In this case, the JSP page is expecting `message` to be a valid attribute to which we assigned a value in one of our controllers, which we in fact did.

2.5 Tell Spring where to find our Controller class and JSP page

- As you may have noticed earlier, we used the Java annotation `@Controller` to define a class as a controller. That' s convenient and short, but we need to tell Spring where to find the classes with annotations. We do so by first creating a Spring configuration `spring-configuration.xml` within the folder WEB-INF. Let's the following content to that file, which is just the

standard preambles used in Spring configuration files. This is done by adding the following instructions to our configuration file.

```
<?xml version="1.0" encoding="UTF-8"?>
<beans xmlns="http://www.springframework.org/schema/beans"
 xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
 xmlns:context="http://www.springframework.org/schema/context"
 xmlns:mvc="http://www.springframework.org/schema/mvc"
 xsi:schemaLocation="
  http://www.springframework.org/schema/beans
      http://www.springframework.org/schema/beans/spring-beans.
    xsd
      http://www.springframework.org/schema/context
      http://www.springframework.org/schema/context/spring-
    context.xsd
      http://www.springframework.org/schema/mvc
         http://www.springframework.org/schema/mvc/spring-mvc.
    xsd">

 <!-- Spring Configuration  -->

</beans>

```

- As you can see, our configuration is empty. We can tell Spring to look Java Annotations by adding the following right below the comment "Spring Configuration".

```
  <!-- Step 3: Add support for component scanning -->
  <context:component-scan
  base-package="edu.deakin.sit218.coachwebspringapp"
  />

  <!-- Step 4: Add support for conversion, formatting and
     validation
  support
  -->
  <mvc:annotation-driven/>
</beans>

```

The first instruction tell Spring to scan the classes within the package `edu.deakin.sit218.coachwebspringapp`. The second instruction is saying that we will configure Spring, such as Controllers and Views, by means of annotations.

- Tell Spring where to find our view pages by adding the following:

```
<!-- Step 5: Define Spring MVC view resolver -->
<bean
 class="org.springframework.web.servlet.view.
    InternalResourceViewResolver">
 <property name="prefix" value="/WEB-INF/view/" />
 <property name="suffix" value=".jsp" />
</bean>

```

The "prefix" property tells in which folder we are storing our views. The "suffix" property instructs Spring to append the .jsp extension to the view names given by the controllers.

2.6 Create a home or index static page

The last step is to add the home page to our web app. In the previous version this was just a static web page with extension .html. We will keep the same page here, but use extension .jsp.

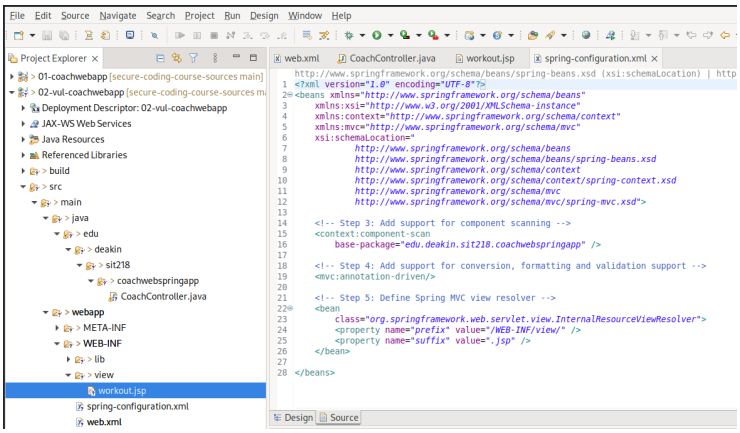

Figure 4: Folder structure containing config files, views and controller class

2.7 Injection attacks

Injection attack is the third most attack type that web application are vulnerable to. Injecting malicious data or input values with an intention to cause harm to the system is source of many attack types and cross-site scripting is one of most popular of them. According to CWE-79[2] the common sources of the cross-site scripting attack are: Cross-site scripting (XSS) vulnerabilities occur when:

- Untrusted data enters a web application, typically from a web request.

[2]https://cwe.mitre.org/data/definitions/79.html

- The web application dynamically generates a web page that contains this untrusted data.
- During page generation, the application does not prevent the data from containing content that is executable by a web browser, such as JavaScript, HTML tags, HTML attributes, mouse events, Flash, ActiveX, etc.
- A victim visits the generated web page through a web browser, which contains malicious script that was injected using the untrusted data.
- Since the script comes from a web page that was sent by the web server, the victim's web browser executes the malicious script in the context of the web server's domain.
- This effectively violates the intention of the web browser's same-origin policy, which states that scripts in one domain should not be able to access resources or run code in a different domain

Let's see how this attack occurs and understand the what part of our code made it possible. First we ask the visitor/student to enter their name and then use random function to generate a response. In workout.jsp page the visitor will see their name and the coach's message. The `@RequestParam` annotation is used to extract query parameters that the user submits. Then the `model.addAttribute("name",studentName)` method is used to send the user's name to the final display page we will show to the user i.e workout page.

Step1: In the index page we are going to ask the user to enter their name.

```
<!DOCTYPE html>
<html>
<head>
<meta charset="ISO-8859-1">
<title>SIT218 Secure Coding</title>
</head>
<body>
<h1>Workout consultation with experts
<br>
<br></h1>

<form action = "workout" method = "Get">
<input type ="text" name = "studentName" placeholder="What's your name
    ?"/>
<input type ="submit"/>
</form>

</body>
</html>
```

Step2: In the controller class we are going to fetch the user's name and then return a random workout (coach's message) to the user via the workout page

Figure 5: Web page asking user to enter their name

```
1
2   @Controller
3   @RequestMapping("/workout")
4   public class CoachController {
5
6    @GetMapping
7    public String workout(@RequestParam("studentName") String name,
      Model model) {
8    model.addAttribute("name", name);
9    Random r = new Random();
10   if (r.nextBoolean())
11   model.addAttribute("message", "No training today");
12   else
13   model.addAttribute("message", "Spend 30 min running");
14
15   return "workout";
16   }
17
```

Step3: Change the workout page to show the user's name and the coach's message

```
1
2 <%@ page language="java" contentType="text/html; charset=ISO-8859-1"
     pageEncoding="ISO-8859-1"%>
3 <!DOCTYPE html>
4 <html>
5 <head>
6 <meta charset="ISO-8859-1">
7 <title>First spring webcoach-app</title>
8 </head>
9 <body>
10 <h1>Coach's Message</h1>
11 <p>Hi ${name}!</p>
12 <p>${message}</p>
13
14 </body>
15 </html>
16
17
```

This is the index page shown to the user
Enter a name and a click submit.

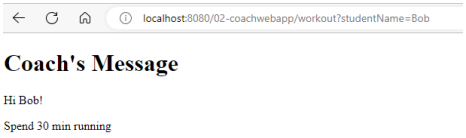

Figure 6: Response after clicking submit

Figure 7: Response with a HTML code as input

2.7.1 Attack:

Not all users have good intentions or some make mistakes and enter input that can cause harm. In the text box enter `<strong>Bob</strong>`. What type of input are you giving? What difference do you see in the output. Now try with `<h1>Bob</h1>`

you might have noticed the change now, so you have just injected a HTML code and controlled how the output looks.

Now lets inject more dangerous input types. Most websites use JavaScript to provide a rich internet application to end users. But these JavaScript function calls can be compromised to reveal more website information and control its behaviour.

Insert the following input into the name field and click submit.

```
1
2 <script>alert("XSS on CoachwebAPP");</script>
3
4
```

2.7.2 Vulnerable Code

In our application, index page receives the input from the user, controller class processes the data and workout page shows the final output to the user. The vulnerable part of code is where we get the user input **studentName** and store the user input in a variable **name** and inject this data into the model using

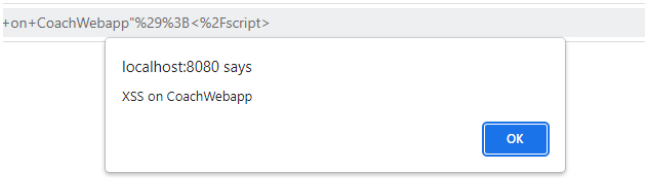

Figure 8: Alert generated by our injected code

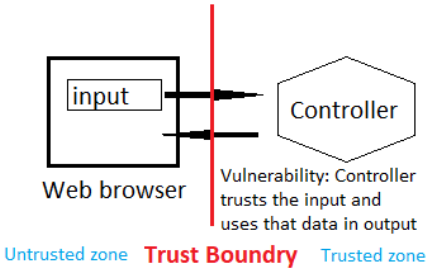

Figure 9: Reflected XSS

model.addAttribute("name",name) and in the workout page we display this value in the coach's message **$name**. Essentially reflecting the untrusted user input back to the user's browser to execute as shown in figure 9.

Places in the web application where you can commonly inject code are but not limited to:

- Search fields which echo input search string
- Input fields that print the user supplied data
- Error messages that show user-supplied text/data
- Hidden fields[3] that contain user supplied data
- Message boards showing user messages
- Comments forms showing user comments
- HTTP Headers

Attacks that take advantage of this vulnerability are reflected XSS exploit which occur when an attacker forces the victim to supply dangerous content to a vulnerable web application, which is then reflected back to the victim

[3]https://www.w3schools.com/tags/att_input_type_hidden.asp

and executed by the web browser. The most common mechanism for delivering malicious content is to include it as a parameter in a URL that is posted publicly or e-mailed directly to the victim.

There are various types of XSS (source https://cwe.mitre.org/data/definitions/79.html):

- **Reflected XSS**: The server reads data directly from the HTTP request and reflects it back in the HTTP response. URLs constructed in this manner constitute the core of many phishing schemes, whereby an attacker convinces a victim to visit a URL that refers to a vulnerable site. After the site reflects the attacker's content back to the victim, the content is executed by the victim's browser.

- **Stored XSS**: The application stores dangerous data in a database, message forum, visitor log, or other trusted data store. At a later time, the dangerous data is subsequently read back into the application and included in dynamic content. From an attacker's perspective, the optimal place to inject malicious content is in an area that is displayed to either many users or particularly interesting users. Interesting users typically have elevated privileges in the application or interact with sensitive data that is valuable to the attacker. If one of these users executes malicious content, the attacker may be able to perform privileged operations on behalf of the user or gain access to sensitive data belonging to the user. For example, the attacker might inject XSS into a log message, which might not be handled properly when an administrator views the logs.

- **DOM-based XSS**: In DOM-based XSS, the client performs the injection of XSS into the page; in the other types, the server performs the injection. DOM-based XSS generally involves server-controlled, trusted script that is sent to the client, such as Javascript that performs sanity checks on a form before the user submits it. If the server-supplied script processes user-supplied data and then injects it back into the web page (such as with dynamic HTML), then DOM-based XSS is possible.